

# Samprada Gifts

## Project handoff and reference — everything needed to resume work

This document is a complete snapshot of the Samprada Gifts e-commerce project: what was built, where it lives, what state the database is in, what to do next, and the decisions and gotchas worth knowing. It was written so that work can continue with no other context. Generated 17 August 2026.

### Status at a glance

| Item           | State                                                                      |
|----------------|----------------------------------------------------------------------------|
| Code           | All committed AND pushed to GitHub. Nothing is at risk.                    |
| Branch         | feat/storefront-frontend (0 unpushed commits)                              |
| Repository     | github.com/jsrkrishna999-cmyk/samprada-gifts (private)                     |
| Storefront     | Working. 86 products served live from Supabase.                            |
| Admin panel    | Built and in the codebase. Needs an admin user created (5 minutes).        |
| Database       | Supabase free tier. schema + migrations 002 and 003 all applied.           |
| Not yet loaded | 60 new photos (siva-catalog.sql) - would take the catalog to 146 products. |
| Payments       | Not built. Orders record a payment method but collect no money.            |
| Deployment     | Not deployed. Runs locally only.                                           |

### The single most important fact

**All work is committed and pushed to GitHub.** If this machine is lost: clone the repo, create a .env.local (section 3), run npm install, and everything is back. The only things NOT in git are the .env.local secrets and the original photo zip.

# 1. What this project is

A premium Indian traditional return-gifts storefront for Samprada Gifts - a real business selling favours for weddings, Haldi Kumkum, Varalakshmi Vratham, Gruhapravesam, baby showers, Navaratri, Diwali and corporate gifting. Based in Vijayawada, Andhra Pradesh.

## Stack

| Layer           | Choice                              | Notes                                              |
|-----------------|-------------------------------------|----------------------------------------------------|
| Framework       | Next.js 16 (App Router), TypeScript | Server Components for data, Client for interaction |
| Styling         | Tailwind CSS v4, Framer Motion      | @theme inline syntax; brand tokens in globals.css  |
| Database + Auth | Supabase (free tier)                | Postgres with Row Level Security                   |
| Images          | Local repo files + Supabase Storage | Migration script written, not yet run              |
| Hosting         | Vercel (planned, free tier)         | Not deployed yet                                   |
| Payments        | Razorpay (planned)                  | Not started                                        |

## Brand palette

```
maroon-500 #7A1F38 gold #C9972F cream #FFF8F0
maroon-600 #651A2E gold-light #E3C27A ivory #FFFDF6
maroon-900 #2F0B16 gold-dark #A5771F brown #3A2418
temple-red #A3182E sandalwood #E0AA71
```

Fonts: Playfair Display (headings) and Inter (body), both loaded via next/font.

# 2. Where everything lives

| What               | Where                                                                                                                             |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Code               | /Users/localaitv/Documents/sampradaya-gifts                                                                                       |
| GitHub             | <a href="https://github.com/jsrkrishna999-cmyk/samprada-gifts">https://github.com/jsrkrishna999-cmyk/samprada-gifts</a> (private) |
| Supabase project   | <a href="https://ftslkpmlijbccrlvsugct.supabase.co">https://ftslkpmlijbccrlvsugct.supabase.co</a>                                 |
| Supabase dashboard | supabase.com, project Samprada_gifts, org jsrkrishna999-cmyk                                                                      |
| Photo sources      | Google Sheet 'PRODUCTS LIST' (54 items) and a zip of 60 WhatsApp photos                                                           |
| Business contact   | jsrkrishna999@gmail.com / +91 8688085332 / Vijayawada                                                                             |

# 3. Environment variables (.env.local)

This file is git-ignored and must be recreated by hand on a new machine. The first two values are safe to expose publicly - Supabase's Row Level Security is what protects the data, not secrecy of these keys.

```
NEXT_PUBLIC_SUPABASE_URL=https://ftslkpm1jbccrlvsugct.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_eE9Bb54VEMOB2hvSn4s0pw_NsBEKUud
NEXT_PUBLIC_SITE_URL=http://localhost:3000
```

```
# Only needed by the image-migration script. NEVER prefix with NEXT_PUBLIC_.
SUPABASE_SECRET_KEY=sb_secret_... (Supabase dashboard > Settings > API)
```

**Important:** on deploy, change NEXT\_PUBLIC\_SITE\_URL to the real https domain. It builds Open Graph image URLs and share links - left as localhost, every shared link points at your laptop.

## 4. Database state

Verified live on 17 August 2026 by querying the Supabase REST API.

| SQL file                                     | Purpose                                                   | Applied? |
|----------------------------------------------|-----------------------------------------------------------|----------|
| supabase/schema.sql                          | categories, products, reviews + public-read RLS           | YES      |
| supabase/seed.sql                            | 32 demo products (invented, stock photos)                 | YES      |
| supabase/migrations/002_real_catalog.sql     | nullable prices, status and source columns, product_costs | YES      |
| supabase/real-catalog.sql                    | 54 real products + 6 categories from the sheet            | YES      |
| supabase/migrations/003_admin_and_orders.sql | admins, orders, order_items, storage bucket, RLS          | YES      |
| supabase/siva-catalog.sql                    | 60 more products from the photo zip                       | NOT RUN  |

### Live contents right now

| Table               | Rows | Detail                                                                                          |
|---------------------|------|-------------------------------------------------------------------------------------------------|
| products            | 86   | 32 with source='demo', 54 with source='catalog'                                                 |
| categories          | 14   | 8 demo + 6 real: deity-favors, hanging-favors, animal-motifs, jute-bags, thamboolam-sets, dolls |
| admins              | 0    | EMPTY - this is why /admin is not accessible yet                                                |
| orders, order_items | 0    | No orders placed yet                                                                            |
| product_costs       | 54   | Supplier cost prices. Private: admin-only read.                                                 |

### Key schema concepts

| Column          | Meaning                                                                                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| products.status | 'active' (buyable, has a price), 'price_on_request', or 'coming_soon'. Non-active products show no price and an Enquire button instead of Add to Cart.                        |
| products.source | 'demo' (invented), 'catalog' (from the sheet), 'catalog-siva' (from the zip). Lets each import be replaced independently, and lets the demo rows be deleted in one statement. |
| products.price  | NULL means no retail price set. A database constraint forbids status='active' with a null price.                                                                              |
| product_costs   | Separate table, RLS on, NO public read policy. Supplier costs must never reach the browser.                                                                                   |

## 5. Do these next, in order

### Step 1 - Become an admin (5 minutes, unblocks the admin panel)

The admin panel is fully built, but the admins table is empty, so /admin redirects to login.

- Run npm run dev, go to /signup, register with your real email, confirm via the emailed link.
- In the Supabase SQL Editor, run the statement below using that same email.
- Go to /admin - dashboard, product editor, photo upload and orders are all there.

```
insert into admins (user_id, email)
select id, email from auth.users where email = 'YOUR@EMAIL.COM'
on conflict (user_id) do nothing;
```

### Step 2 - Load the 60 new products

The photos are already optimised and committed; only the SQL is outstanding. Paste supabase/siva-catalog.sql into the Supabase SQL Editor and run it. This takes the catalog from 86 to 146 products and adds a 'Haldi Kumkum Favors' category.

### Step 3 - Set real prices

All 114 real products are 'price\_on\_request' - browsable but not buyable. The figures in the original sheet (Rs 3.50 to Rs 60) are SUPPLIER COST prices and are deliberately never published; they live in the private product\_costs table. Set retail prices in /admin/products, which has inline editing - saving a price flips a product to 'active' automatically.

### Step 4 - Write real descriptions

Every real product currently carries a generated placeholder: 'X is a handcrafted [type] from the Samprada Gifts collection (product code C0XX).' Fine as scaffolding, weak for sales and SEO.

### Step 5 - Delete the demo products before taking real orders

32 invented products (Lotus Haldi Kumkum Plate, Wedding Hamper Deluxe and so on) are live and buyable with stock photos. A customer could order something that does not exist. One statement removes them - which is exactly why they were tagged with a source.

```
delete from products where source = 'demo';
```

### Step 6 - Deploy to Vercel

- Import the GitHub repo at vercel.com. The framework auto-detects as Next.js.
- Add the three NEXT\_PUBLIC\_ variables, with NEXT\_PUBLIC\_SITE\_URL set to the real domain.
- After deploying, test a shared link with Facebook's Sharing Debugger to force a preview re-scrape. WhatsApp caches previews aggressively, so a bad first fetch can stick.

### Step 7 - Payments (Razorpay)

Not started. Today an order records a payment method but collects no money; 'Paid' is a status set by hand in admin, and the revenue figure counts only orders marked paid.

## 6. What was built

| Area               | What exists                                                                                                                                                                                                                                          |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Storefront         | Home, Shop (filters and sort), Categories, Category, Product detail, Cart, Wishlist, Checkout, Order success, Orders, Account, Login, Signup, About, Contact, FAQ, Privacy, Terms                                                                    |
| Auth               | Real Supabase Auth: signup with email confirmation, login, logout, cookie session refreshed by middleware. Replaced an earlier localStorage demo login.                                                                                              |
| Data layer         | Server Components read live from Supabase per request (src/lib/supabase/queries.ts). Client components use an in-memory catalog cache (src/context/catalog-context.tsx).                                                                             |
| Admin panel        | /admin dashboard (orders, revenue, customers, unpriced count); /admin/products (search, filter, inline price edit); /admin/products/[slug] (full edit, photo upload, cost and margin); /admin/orders plus detail (status updates); /admin/categories |
| Orders             | Real orders written at checkout into orders and order_items. Customers see their own; admins see all.                                                                                                                                                |
| Sharing            | Open Graph and Twitter cards on product, category and home pages. ShareButton uses the native mobile share sheet, falling back to WhatsApp / Facebook / copy-link. Placed on the product page, wishlist and order-success page.                      |
| Shareable wishlist | /wishlist?items=slug1,slug2 renders a read-only shortlist with a 'Save all to my wishlist' button - built for families choosing favours together.                                                                                                    |
| Logos              | Five concepts in public/logo/: diya, lotus, kalasham, S monogram, temple arch. None chosen yet; the site still uses the old placeholder icon.                                                                                                        |

## 7. Scripts

| Command                                                                    | What it does                                                                                             |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| npm run dev                                                                | Start the dev server on port 3000                                                                        |
| npm run build                                                              | Production build - use to catch errors before deploying                                                  |
| npx tsc --noEmit                                                           | Type-check only                                                                                          |
| node scripts/generate-og-images.mjs                                        | Rebuild the 1200x630 WhatsApp share cards for every product photo                                        |
| node scripts/import-siva-batch.mjs                                         | Re-run the 60-photo import. Edit the ITEMS list first to fix names.                                      |
| npx tsx scripts/generate-real-catalog-sql.ts                               | Regenerate the 54-product SQL from the sheet data                                                        |
| node --env-file=.env.local scripts/migrate-images-to-storage.mjs --dry-run | Preview moving all 114 photos into Supabase Storage. Drop --dry-run to apply. Needs SUPABASE_SECRET_KEY. |

## 8. Decisions made, and why

These were judgment calls. Any can be reversed - knowing the reasoning makes that easier.

| Decision                                                      | Reasoning                                                                                                                                                                                      |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Supplier prices are never published                           | The sheet's Rs 3.50-60 figures are wholesale costs. Publishing them would hand competitors your margins. They live in <code>product_costs</code> , which has RLS on and no public read policy. |
| Real products added alongside demo rather than replacing them | Your explicit choice. The trade-off is that invented products are currently buyable - see Step 5.                                                                                              |
| Six new categories invented for the real stock                | Ganesh hangers and jute bags do not fit demo categories like 'Diwali'. The mapping lives in <code>scripts/generate-real-catalog-sql.ts</code> and is easy to change.                           |
| No secret key anywhere in the app                             | Admin writes go through the logged-in user's own session, gated by RLS checking the <code>admins</code> table. The secret key is needed only by the one-off image-migration script.            |
| Checkout re-reads prices server-side                          | The browser cart is treated as untrusted, so a tampered cart cannot create an order at a price you never set.                                                                                  |
| Share cards are pre-rendered JPEGs                            | Catalog photos are WebP, which WhatsApp renders unreliably, and they are small and portrait. Each product gets a 1200x630 JPEG card so previews always look right.                             |
| Wordmark is HTML text, not baked into the logo SVG            | An SVG used as an <code>img</code> cannot inherit Playfair Display from the page, so a baked wordmark would silently fall back to a different serif.                                           |

## 9. Known gaps and gotchas

| Issue                              | Detail                                                                                                                                                                                                                        |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Demo products are buyable          | 32 invented products can be added to a cart and ordered. Delete before going live.                                                                                                                                            |
| Placeholder descriptions           | All 114 real products carry generated boilerplate descriptions.                                                                                                                                                               |
| No payment collection              | Razorpay is not integrated. 'Paid' is set manually in admin.                                                                                                                                                                  |
| <code>SITE_URL</code> is localhost | Must change on deploy, or every shared link and <code>og:image</code> points at your laptop.                                                                                                                                  |
| Photos are developer-managed       | The 114 imported photos are repo files, so they cannot be changed from <code>/admin.scripts/migrate-images-to-storage.mjs</code> fixes this and has not been run.                                                             |
| Admin uploads get no share card    | Cards are pre-built and matched by slug. A photo uploaded through <code>/admin</code> falls back to the raw file; if it is WebP its WhatsApp preview may not render.                                                          |
| Some product names are guesses     | The 60 zip photos were named by looking at them. Least certain: SV026 'Satyanarayana Swamy', SV036 'Jagannath', SV038 'Painted Oval', SV049 'Traditional Lady'.                                                               |
| Three pre-existing lint errors     | <code>setState</code> -inside-effect warnings in <code>cart-context</code> , <code>wishlist-context</code> and <code>OrderSuccessContent</code> , from <code>localStorage</code> hydration. Harmless; they predate this work. |
| Branch not merged to main          | All work sits on <code>feat/storefront-frontend</code> . It was never decided whether to open a pull request or make it the default branch.                                                                                   |

## 10. A trap worth remembering

The Supabase table types in `src/lib/supabase/types.ts` MUST be declared with **type** aliases, not **interface**. Supabase constrains each table to `Record<string, unknown>`, and a TypeScript interface has no implicit index signature - so using one silently collapses every table type to 'never' and produces around a hundred confusing errors across unrelated files. This cost real debugging time once already. If you ever regenerate those types by hand, keep them as type aliases.

End of handoff. Everything above is verifiable from the repository itself - the SQL files, scripts and code comments carry the same reasoning.